TryHackMe — Computer Fundamentals Module

Client-Server Basics

Learning Notes — The Model, Protocols, DNS, Ports & HTTP in Practice

> ***"Just like in society, where people distinguish themselves with a particular set of skills and offer these as a service, interconnected computer systems started to specialise — and the client-server model was born."***

Client-Server Model | DNS & IP | Ports | HTTP(S) | GET in Practice

1. The History — From Isolated Computers to the Internet

Initially, most computers worked alone: they stored their own files, ran their own programs, and did not communicate with other computers. As technology advanced, multiple organisations around the world began interconnecting these isolated systems to enable information exchange and resource sharing regardless of physical distance.

1.1 The Early Networks That Built the Internet

Network	Contribution
ARPANET	The first wide-area packet-switching network (1969), funded by the US Department of Defense. Designed to survive partial network failure. The direct ancestor of the modern internet.
CYCLADES	French research network (1970s). Pioneered the concept of putting reliability responsibility on end hosts rather than the network — a key principle of TCP/IP.
NPL	UK National Physical Laboratory network. Contributed foundational packet-switching theory and early experimental implementations.
NSFNET	National Science Foundation Network (1980s). Replaced ARPANET as the backbone of the US internet and opened it to academic use, driving rapid growth.

The Core Insight

Just as people in society distinguish themselves with skills and offer them as services, interconnected computer systems started to specialise. Some became servers offering resources; others became clients consuming them. This specialisation is the foundation of every networked service you use today — websites, email, file sharing, streaming, gaming.

2. The Client-Server Model

The client-server model is the architectural framework that defines how computers communicate and provide services over a network. It is the foundation of the internet and virtually every networked application — from web browsers to email clients to online games.

2.1 The Pizza Analogy — Alice, Bob, and Luigi's

The room introduces the client-server model through a memorable pizza ordering analogy that makes the abstract concepts concrete:

Pizza Analogy	Pizza World	Computer World
Alice	The customer — places the order, initiates the interaction	The client — initiates every request (browser, app, email client)
Bob	The delivery person — carries the request to Luigi's and back	The network / intermediary — carries data between client and server
Luigi's	The restaurant / kitchen — receives, processes, and fulfils order	The server — receives requests, processes them, returns responses
The menu	Defines what can be ordered — a shared language for requests	The protocol — defines how requests must be formatted (HTTP, FTP, etc.)
The order	Formatted correctly per the menu, sent to Luigi's	The HTTP request — correctly formatted per the protocol specification
Response	Food delivered — or an error (no pepperoni, can't understand)	HTTP response — content returned, or error code (404, 500, etc.)

The Critical Rule: The Client Always Initiates

In the client-server model, the client ALWAYS initiates the request. The server ALWAYS waits and responds. A server never spontaneously pushes data to a client unless the client first established a connection and is listening for updates. This is a fundamental rule — and understanding it is key to understanding how attacks work (e.g. getting a server to initiate a connection back to the attacker requires exploiting this model).

2.2 What Is a Protocol?

When Alice formulates her request to Bob, she uses the language Luigi's Pizza understands — and uses the menu to determine what she can order. In computer terms, this 'language and menu' is called a protocol: a set of rules that defines how messages must be formatted, what requests are valid, and how responses should be structured.

Protocol Feature	What It Defines
Message format	How a request or response must be structured — headers, body, encoding (like filling in a form correctly).
Valid commands	What actions are permitted — GET, POST, DELETE for HTTP; CONNECT, LIST, RETR for FTP.
Error handling	What to send when something goes wrong — status codes (404 Not Found, 500 Server Error).
Authentication	How identity is verified before privileged actions are permitted.
State management	Whether the protocol remembers previous interactions (stateful) or treats each independently (stateless).
Port assignment	Which TCP/UDP port the service listens on by default — HTTP=80, HTTPS=443, SSH=22, DNS=53.

3. DNS — The Internet's Address Book

When Alice sent Bob her request, only the name of the pizza place was known. With that alone, it is not possible to reach the destination. So Bob entered Luigi's Pizza into a GPS device, and the device returned the coordinates. DNS works the same way: it translates a domain name (tryhackme.com) into the server's IP address so the browser knows where to connect.

Room Answer: What do we call the address of a server?

Internet Protocol address (IP address). Room Answer: What do we use to identify a specific service on a server? Port.

3.1 IP Addresses — The Location Coordinates of the Internet

An IP address is the unique numerical address assigned to every device on a network. Think of it as the address to your home: street name, house number, postal code, city, and country — but for computer systems. Every server hosting a website, every router, and every connected device has one.

- IPv4: four octets of decimal numbers (e.g. 104.26.10.229) — provides ~4.3 billion unique addresses.

- IPv6: eight groups of hexadecimal (e.g. 2606:4700:20::681a:be5) — provides vastly more addresses for the growing internet.
- 127.0.0.1 is the loopback address — it always refers to your own machine ('localhost'). Used in the room's practical lab.
- IP addresses route data across networks (Layer 3). They tell the network where to send packets globally.

3.2 DNS Resolution — GPS for the Internet

1 **Enter domain name** *(You → Browser)*
You type 'httpdemo.local:8080' into Firefox. The browser needs an IP address — not a name — to connect.

2 **DNS query** *(Browser → DNS Server)*
The browser queries a DNS server: 'What is the IP address for httpdemo.local?' DNS resolves the name to 127.0.0.1 (the local machine in this lab).

3 **IP address returned** *(DNS → Browser)*
DNS returns the IP address. The browser now knows the destination. On the internet, this would be the actual server IP (e.g. 104.26.10.229).

4 **TCP connection + HTTP request** *(Browser → Server)*
Browser connects to the IP address on the specified port (8080 in the lab, 80 for HTTP, 443 for HTTPS). Sends the HTTP GET request.

5 **HTTP response** *(Server → Browser)*
Server processes the request and returns the response: status code (200 OK) + response headers + HTML body. Browser renders the page.

4. Ports — Directing Traffic to the Right Service

An IP address gets data to the right device. A port number gets data to the right service on that device. One server can run dozens of different services simultaneously — each listening on a different port number. Ports are how the OS knows which application should receive incoming data.

The Apartment Building Analogy

Think of an IP address as the building address and a port as the apartment number. The postal carrier (network) delivers to the building (IP address), and the apartment number (port) identifies which resident (service) receives the delivery. Without a port, you know the building but not who to knock on.

4.1 Well-Known Port Numbers

Port	Protocol	Transport	Service Description
20/21	FTP	TCP	File Transfer Protocol — transfers files. Port 21 = control; Port 20 = data.
22	SSH	TCP	Secure Shell — encrypted remote terminal access. Replaces insecure Telnet.
23	Telnet	TCP	Legacy unencrypted remote terminal. Deprecated — sends passwords in plain text.
25	SMTP	TCP	Simple Mail Transfer Protocol — sending email between mail servers.
53	DNS	UDP/TCP	Domain Name System — resolves domain names to IP addresses.
80	HTTP	TCP	HyperText Transfer Protocol — unencrypted web traffic.
110	POP3	TCP	Post Office Protocol 3 — downloading email from a server.
143	IMAP	TCP	Internet Message Access Protocol — syncing email across devices.
443	HTTPS	TCP	HTTP Secure — encrypted web traffic using TLS. The standard for modern websites.
3389	RDP	TCP/UDP	Remote Desktop Protocol — graphical remote access to Windows systems.
8080	HTTP alt	TCP	Alternative HTTP port — used in the room's practical lab (httpdemo.local:8080).

Security Note — Port Scanning

Attackers use port scanners (primarily Nmap) to probe all 65535 ports on a target and discover which services are running. Each open port is a potential attack vector. Security hardening requires: closing unnecessary ports, changing default ports for sensitive services (SSH from 22 to a non-standard port), and implementing firewalls to restrict which IPs can reach each port.

5. HTTP(S) — The Protocol of the Web

HTTP (HyperText Transfer Protocol) is the stateless client-server protocol used for the World Wide Web. Stateless means each request is processed independently — the server retains no memory of previous requests. HTTPS adds TLS encryption to HTTP, protecting the data in transit from eavesdroppers.

Stateless but Not Sessionless

Although HTTP itself is stateless, modern web applications introduce statefulness at the application level. When you log into a website, the server creates a session identifier stored in a cookie or token and sent with each subsequent request. Without this mechanism, you would need to authenticate on every single page — because the server has no memory of your previous login. The cookie carries the state that HTTP cannot.

5.1 The 9 HTTP Methods (RFC-Defined Commands)

In HTTP specifications (called RFC documents — Request for Comments), there are 9 core commands. In HTTP terminology, these are called methods rather than commands:

Method	CRUD Operation	Purpose
GET	Read	Retrieves a resource from the server. The most common method — used every time a browser loads a page. Examined in detail in the room's practical lab.
POST	Create	Sends data to the server to create a new resource — login forms, sign-up, file uploads, API calls.
PUT	Update	Replaces the entire resource at the target URL with the data in the request body.
PATCH	Update	Partially updates a resource — modifies only the specified fields, unlike PUT which replaces everything.
DELETE	Delete	Removes the specified resource from the server.
HEAD	Read	Identical to GET but returns only headers — no response body. Used to check if a resource exists.
OPTIONS	Info	Returns the HTTP methods supported for a specific URL. Used in CORS preflight requests.
TRACE	Debug	Loop-back diagnostic — echoes the received request. Often disabled due to cross-site tracing risks.
CONNECT	Tunnel	Establishes a TCP tunnel through an HTTP proxy — used for HTTPS connections through a proxy server.

6. Practical Lab — Inspecting a Real HTTP GET Request

The room includes a hands-on lab where a Virtual Machine runs a local web server at httpdemo.local:8080. Using Firefox Developer Tools (Network tab), you inspect real HTTP request and response traffic — seeing exactly what happens when a browser makes a GET request.

6.1 Opening Firefox Developer Tools

- Click the Firefox icon on the VM Desktop — the browser opens to http://httpdemo.local:8080.
- Press F12 (or right-click → Inspect) to open Firefox Developer Tools.
- Click the 'Network' tab in Developer Tools — this shows all HTTP requests and responses.
- Reload the page (click the circular refresh icon) — multiple GET requests appear in the Network tab.
- Click on any request to see its details in the right panel.

6.2 Anatomy of the GET Request — What the Lab Shows

```
# GET Request sent by Firefox to httpdemo.local:8080
GET / HTTP/1.1
Host: httpdemo.local:8080
User-Agent: Mozilla/5.0 (Firefox/87.0)
Accept: text/html,application/xhtml+xml

# The '/' path translates to 'index.html' — the default page
# The Host header tells the server which site is being requested
```

6.3 The Request Detail Fields — Developer Tools Right Panel

Field Name	Example Value	What It Tells You
Scheme	`http`	Which protocol was used — HTTP or HTTPS. Determines whether the connection is encrypted.
Host	`httpdemo.local`	The name of the server being requested. The Host header enables virtual hosting (many sites per server).
Filename	`/`	Which resource was requested from the server. The path '/' translates to 'index.html' — the default page.

Address	`127.0.0.1`	The IP address where the website is hosted. In the lab, 127.0.0.1 means the site is hosted on the same machine (loopback/localhost).
Status	`200 OK`	Whether the request was successful. 200 OK means the server found and returned the resource. Other codes indicate errors.

6.4 The HTTP Response — Header and Body

When the server receives the GET request, it sends back a response divided into two parts:

Response Part	Contents & Purpose
Response Header	Metadata about the response — status code (200 OK), Content-Type (text/html), Content-Length (size in bytes), Server software version, Date. Visible in the 'Headers' tab of Developer Tools.
Response Body	The actual content requested — the HTML of the web page, JSON data, image bytes, etc. Visible in the 'Response' tab of Developer Tools. This is what the browser renders.

```
# Example HTTP Response
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Server: Apache/2.4.41

# [blank line — separates headers from body]

<!DOCTYPE html>
<html>
  <head><title>HTTPDemo</title></head>
  <body><h1>Welcome to HTTPDemo</h1></body>
</html>
```

127.0.0.1 — The Loopback Address

In the lab, the IP address shown is 127.0.0.1. This is the loopback address (also called localhost) — it always refers to your own machine. It is used to run and test servers locally without a network connection. It is a fixed address: 127.0.0.1 is always your own computer, on any system in the world.

7. The Complete Client-Server Communication Flow

Combining everything from the room, here is the complete flow of a single web page request — from typing a URL to seeing the page rendered in your browser:

#	Stage	Component	What Happens
1	**URL Input**	Browser (client)	User types 'http://httpdemo.local:8080' — browser parses scheme (http), host (httpdemo.local), port (8080), path (/).
2	**DNS Resolution**	DNS / OS resolver	Browser asks DNS to resolve 'httpdemo.local' → returns 127.0.0.1. Now the client has the destination IP.
3	**TCP Connection**	OS + Network	Browser opens a TCP connection to 127.0.0.1 on port 8080. Three-way handshake: SYN → SYN-ACK → ACK.
4	**HTTP GET Request**	Browser → Web server	Browser sends 'GET / HTTP/1.1' with headers (Host, User-Agent, Accept). Server receives the request.
5	**Server Processing**	Web server (Apache/Nginx)	Server looks up the requested path '/' — maps to index.html. Reads the file from storage.
6	**HTTP Response**	Web server → Browser	Server sends 'HTTP/1.1 200 OK' with headers and the HTML body. Status 200 confirms success.
7	**Sub-requests**	Browser → Server	Browser parses HTML, finds CSS/JS/image references — sends additional GET requests for each linked resource.
8	**Render Page**	Browser	Browser combines all received resources (HTML + CSS + JS + images) and renders the complete visible page.

8. Room Answers — Quick Reference

Question	Answer
What do we use to identify a specific service on a server?	`Port`
What do we call the address of a server?	`Internet protocol address`
What protocol abbreviation is used for web browsing (plain text)?	`HTTP`

What does HTTPS stand for?	`HyperText Transfer Protocol Secure`
What makes HTTP stateless?	`Each request is processed independently — no memory of previous requests`
What does the path '/' in a GET request translate to?	`index.html`
What was the IP address shown in the lab (hosting on same machine)?	`127.0.0.1`
What status code means a request was successful?	`200 OK`
Where is the response body visible in Firefox Developer Tools?	`The Response tab`
What networks preceded and shaped the modern internet?	`ARPANET, CYCLADES, NPL, NSFNET`

9. Security Implications

The Client-Server Model Defines the Attack Surface

Every service a server offers is reachable from the network — and is therefore a potential target. The attack surface is the sum of all ports, protocols, and services exposed by a server. Minimising the attack surface (disabling unused services, closing unnecessary ports, using firewalls) is a fundamental security practice.

HTTP Is Plaintext — HTTPS Is Non-Negotiable

HTTP transmits all data including credentials, session tokens, and sensitive content as unencrypted plaintext. Anyone on the same network (ISP, coffee shop Wi-Fi, compromised router) can read every HTTP message. HTTPS encrypts the entire HTTP exchange with TLS, making interception unreadable. Every modern website must use HTTPS.

Statelessness Creates Session Management Challenges

Because HTTP is stateless, session management must be implemented at the application layer — typically via cookies or tokens. Weak session management is a critical vulnerability: predictable session tokens allow attackers to hijack authenticated sessions (session hijacking); cookies without HttpOnly or Secure flags can be stolen via XSS or network interception.

DNS Is an Unauthenticated Protocol by Default

Standard DNS queries and responses are transmitted in plaintext without authentication. DNS spoofing (cache poisoning) can redirect users to malicious servers by injecting forged DNS responses. DNS-over-HTTPS (DoH) and DNS-over-TLS (DoT) encrypt DNS traffic. DNSSEC adds cryptographic signing to DNS records to prevent tampering.

Every Open Port Is a Potential Entry Point

Attackers use port scanners (Nmap) to discover which ports are open on a target server. Each open port runs a service, and every service has a history of vulnerabilities. Port 22 (SSH) faces constant brute-force attacks. Port 80/443 is targeted by web application attacks. Closing unnecessary ports and keeping services patched are the primary defences.

The Server Header Reveals Technology Stack

The HTTP Server response header (e.g. Server: Apache/2.4.41) reveals the web server software and version. Attackers cross-reference this with CVE databases to find known vulnerabilities for that exact version. Security best practice: suppress or obfuscate the Server header in production configurations.

10. Key Takeaways & Glossary

10.1 Key Takeaways

- The internet evolved from isolated computers to interconnected networks through ARPANET, CYCLADES, NPL, and NSFNET.
- The client-server model: the CLIENT always initiates; the SERVER always waits and responds.
- Protocol = the shared language and rules for communication between client and server.
- DNS = the internet's GPS — translates domain names to IP addresses (Internet Protocol addresses).
- Port = identifies a specific service on a server (like an apartment number in a building).
- 127.0.0.1 = loopback/localhost — always refers to your own machine.
- HTTP is stateless — each request is independent. Cookies/tokens add statefulness at the application level.
- HTTPS = HTTP + TLS encryption. Required for any transmission of sensitive data.
- HTTP has 9 methods: GET (retrieve), POST (create), PUT (replace), PATCH (partial update), DELETE, HEAD, OPTIONS, TRACE, CONNECT.
- A GET request to '/' retrieves index.html — the default web page.
- HTTP response = header (metadata: status, content-type) + body (the actual HTML/data content).
- Firefox Developer Tools → Network tab → inspect all HTTP requests and responses in real time.

10.2 Core Glossary

Term	Definition

Client	The initiating party in the client-server model — makes requests to the server (browser, app, email client).
Server	The responding party — waits for client requests and returns responses with data or services.
Protocol	A set of rules defining how messages must be formatted and exchanged between client and server.
IP Address	The unique numerical address of a device on a network. IPv4: four decimal octets (e.g. 104.26.10.229).
127.0.0.1	The loopback (localhost) address — always refers to the local machine itself.
DNS	Domain Name System — translates human-readable domain names to machine-readable IP addresses.
Port	A numerical identifier (0–65535) directing traffic to the correct service on a device.
HTTP	HyperText Transfer Protocol — stateless protocol for web communication. Port 80. Unencrypted.
HTTPS	HTTP Secure — HTTP with TLS encryption. Port 443. Required for secure web communication.
Stateless	Each HTTP request is independent — the server retains no memory of previous requests.
Session	Statefulness layer added above HTTP via cookies/tokens — enables 'staying logged in'.
GET	HTTP method — retrieves a resource from the server without modifying server state.
POST	HTTP method — submits data to create or process a resource.
RFC	Request for Comments — official internet standards documents defining protocol specifications (e.g. HTTP RFC).
Response Header	Metadata section of HTTP response — status code, content type, server info, caching directives.
Response Body	Content section of HTTP response — the HTML, JSON, or binary data the client requested.
Status Code	Three-digit code in HTTP response indicating outcome: 200=OK, 404=Not Found, 500=Server Error.
ARPANET	First wide-area packet-switching network (1969) — the direct ancestor of the internet.
Developer Tools	Browser built-in debugging interface (F12) — inspects HTML, Network traffic, Console, and more.
Loopback	Network address (127.0.0.1 / localhost) that routes back to the local machine — used for local testing.

10.3 Next Steps on TryHackMe

- **Immediate:** Continue with the Network Services module — SSH, FTP, SMTP, and other protocols in depth.
- **Essential:** Study the Burp Suite Basics room — intercept, inspect, and modify HTTP requests in real time.
- **Security:** Complete the OWASP Top 10 room — the most critical web application vulnerabilities, all using HTTP.
- **Offensive:** Take the Passive Reconnaissance room — use DNS, HTTP headers, and open ports for OSINT.
- **Applied:** Study Wireshark Basics — see raw HTTP, DNS, and TCP traffic in captured network packets.

Everything Flows Through the Client-Server Model

Every attack on a web application, every network penetration test, every incident response investigation — involves understanding client-server communication. DNS poisoning, HTTP injection, session hijacking, port scanning, MITM attacks — they all exploit or traverse the model covered in this room. Building a deep, intuitive understanding of requests, responses, protocols, and ports makes every subsequent security topic easier to learn.

Source: TryHackMe — Client-Server Basics Room | tryhackme.com/room/clientserverbasics